UNIVERSITY OF THE PEOPLE

Topic: Functional Programming in Java
Student Name: Mubashir Ali
Course: CS 1102-01 - AY2026-T3
Date: March 19, 2026
Assignment Activity: Unit 8

Introduction:

In modern Java programming, handling large datasets efficiently is a critical requirement. The introduction of functional programming concepts in Java 8, particularly the Function interface and Streams API, has significantly improved how developers process and manipulate data. These tools allow developers to write clean, concise, and efficient code while minimizing complexity.

This assignment demonstrates how the Function interface and streams can be used to process a dataset of employees. The program performs various operations such as data transformation, filtering, and aggregation, showcasing the power of functional programming in Java.

Purpose and Characteristics of Function Interface

The Function<T, R> interface is part of the java.util.function package and represents a function that accepts one argument and produces a result.

Key Characteristics

- It is a **functional interface** with a single abstract method apply().
- Supports **lambda expressions**, making code concise.
- Enables **data transformation** in stream pipelines.
- Generic nature, allowing flexibility with different data types.

Usage

The Function interface is commonly used in stream operations such as map() to transform elements.

Example:

```
Function<Employee, String> func = e -> e.getName() + " - " + e.getDepartment();
```

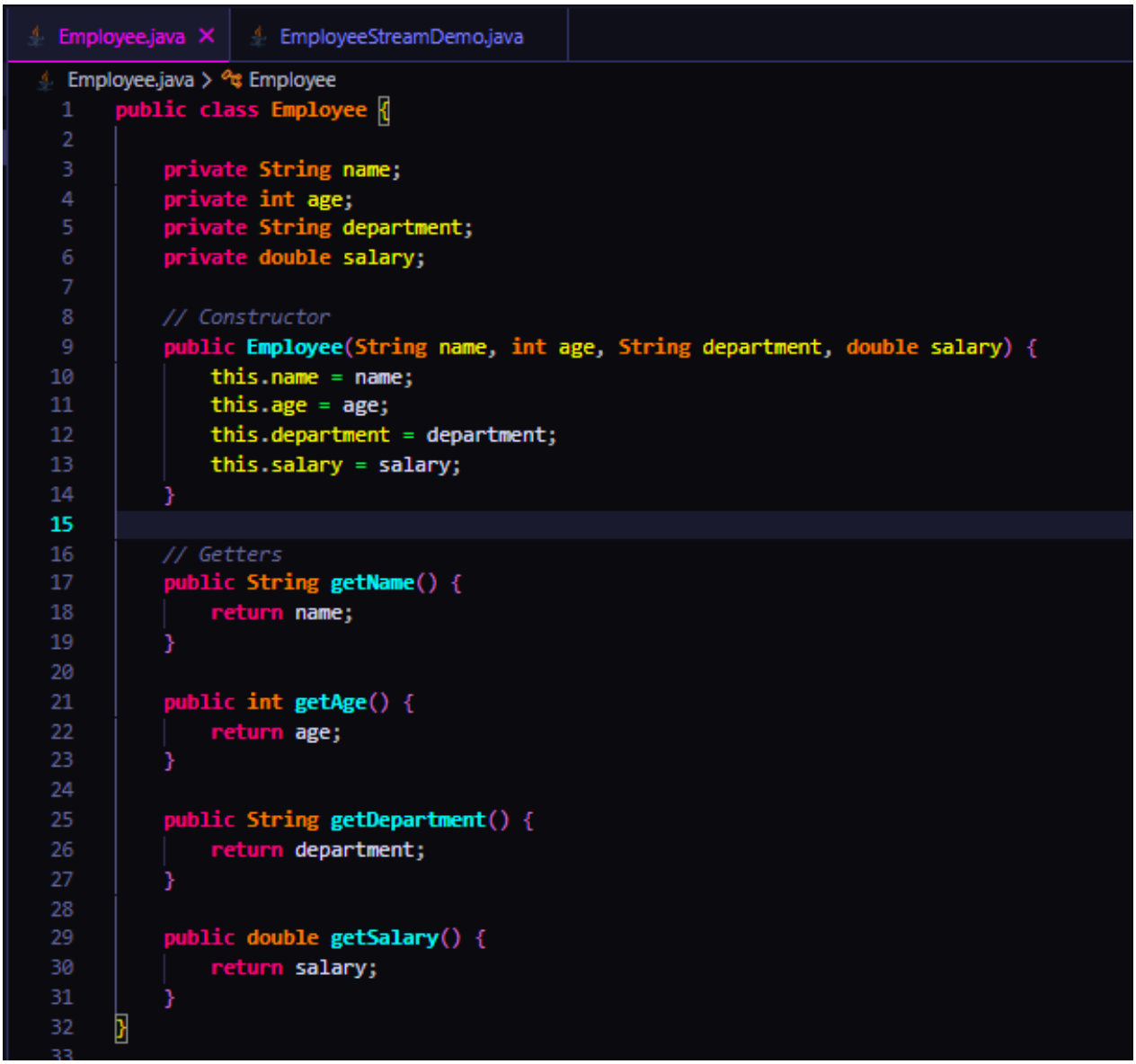
Concept of Streams

Streams in Java provide a modern way to process data collections. Instead of using traditional loops, streams allow developers to perform operations such as filtering, mapping, and aggregation in a declarative manner.

Key Features

- **Pipeline Processing**: Multiple operations can be chained.
- **Lazy Evaluation**: Operations execute only when required.
- **Improved Performance**: Reduces unnecessary computations.
- **Readable Code**: Cleaner and more expressive.

Program – Employee.java



Explanation

The Employee class is a simple model (POJO) that represents an employee entity in a Java program. It stores basic employee details such as name, age, department, and salary using private variables to ensure data encapsulation. The class provides a parameterized constructor to initialize these values when an object is created. Additionally, getter methods are included to allow controlled access to the private fields. This class is mainly used to represent and manage employee data in a structured way, especially when working with collections and stream operations in Java.

Program – EmployeeStreamDemo.java

Employee.java | EmployeeStreamDemo.java

EmployeeStreamDemo.java > EmployeeStreamDemo > main(String[])

```java
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class EmployeeStreamDemo {

    Run | Debug
    public static void main(String[] args) {

        // Dataset stored in collection
        List<Employee> employees = Arrays.asList(
                new Employee(name: "Ali", age: 25, department: "IT", salary: 50000),
                new Employee(name: "Sara", age: 32, department: "HR", salary: 60000),
                new Employee(name: "Ahmed", age: 40, department: "Finance", salary: 70000),
                new Employee(name: "Zara", age: 28, department: "IT", salary: 55000),
                new Employee(name: "Usman", age: 35, department: "Marketing", salary: 65000)
        );

        Function<Employee, String> nameDeptFunction =
                emp -> emp.getName() + " - " + emp.getDepartment();

        // Stream: transform collection
        List<String> nameDeptList = employees.stream()
                .map(nameDeptFunction)
                .collect(Collectors.toList());

        System.out.println(x: "Employee Name & Department:");
        nameDeptList.forEach(System.out::println);

        // Average salary using streams
        double avgSalary = employees.stream()
                .mapToDouble(Employee::getSalary)
                .average()
                .orElse(other: 0);

        System.out.println("\nAverage Salary: " + avgSalary);
```

```
38          // Filter employees age > 30
39          System.out.println(x: "\nEmployees Age > 30:");
40          employees.stream()
41                  .filter(emp -> emp.getAge() > 30)
42                  .forEach(emp ->
43                          System.out.println(emp.getName() + " (" + emp.getAge() + ")")
44                  );
45
46          // Bonus: Highest salary employee
47          employees.stream()
48                  .max(Comparator.comparing(Employee::getSalary))
49                  .ifPresent(emp ->
50                          System.out.println("\nHighest Paid Employee: " + emp.getName())
51                  );
52      }
53  }
```

Program Output

```
D:\UoPeople\2025-26\Term3- Feb-Mar\CS 1102-01 - AY2026-T3\19-MAR>java EmployeeStreamDemo
Employee Name & Department:
Ali - IT
Sara - HR
Ahmed - Finance
Zara - IT
Usman - Marketing

Average Salary: 60000.0

Employees Age > 30:
Sara (32)
Ahmed (40)
Usman (35)

Highest Paid Employee: Ahmed

D:\UoPeople\2025-26\Term3- Feb-Mar\CS 1102-01 - AY2026-T3\19-MAR>
```

Efficiency and Performance

The program efficiently utilizes streams by:

- Avoiding explicit loops and reducing code complexity.
- Using **lazy evaluation**, ensuring operations execute only when needed.
- Applying **short-circuiting methods** like average() and max() for optimized computation.
- Minimizing memory usage through pipeline processing.

Conclusion

This program demonstrates how the Function interface and Streams API can be effectively used to process large datasets in Java. The Function interface provides a flexible way to define transformation logic, while streams enable efficient and readable data processing. By combining these concepts, developers can build scalable and high-performance applications.

References

Oracle. (2024). *The Java™ tutorials: Collections framework*. https://docs.oracle.com/javase/tutorial/collections/

Oracle. (2024). *Package java.util.stream*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/package-summary.html

Oracle. (2024). *Functional interfaces in Java*. https://docs.oracle.com/javase/8/docs/api/java/util/function/package-summary.html

Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.

Herbert, S. (2020). *Java: The complete reference* (11th ed.). McGraw-Hill Education.

Goetz, B. (2006). *Java concurrency in practice*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Schildt, H. (2019). *Java: A beginner's guide* (8th ed.). McGraw-Hill Education.

Oracle. (2024). *Java SE documentation*. https://docs.oracle.com/en/java/javase/